

Chapter 2

JavaScript Variables, Constants & Data Types

This chapter is very important because almost every JavaScript program uses **variables and data types**.

2.1 Learning Objectives

After completing this chapter, students should be able to:

1. Explain what a variable is.
 2. Declare and initialize variables.
 3. Differentiate var, let, and const.
 4. Understand JavaScript data types.
 5. Identify primitive and non-primitive data types.
 6. Use typeof.
 7. Understand undefined and null.
 8. Perform type conversion.
 9. Understand dynamic typing.
 10. Write JavaScript programs using variables and different data types.
-

2.2 What is a Variable?

A **variable** is a named storage location used to hold a value that a program can work with.

For example:

```
let name = "Rahul";
```

Here:

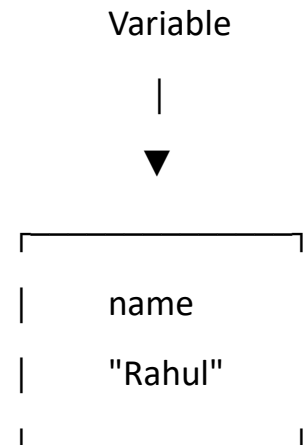
let → keyword

name → variable name

= → assignment operator

"Rahul" → value

We can visualize it as:



Example

let age = 22;

let marks = 85.5;

let name = "Amit";

Three variables have been created:

age → 22

marks → 85.5

name → "Amit"

2.3 Why Do We Need Variables?

Suppose we want to calculate the total marks of a student.

Without variables:

console.log(80 + 75 + 90 + 85);

With variables:

let english = 80;

let java = 75;

```
let dbms = 90;
```

```
let javascript = 85;
```

```
let total = english + java + dbms + javascript;
```

```
console.log(total);
```

Variables make programs:

- Easier to understand
 - Easier to modify
 - Reusable
 - More organized
 - Easier to debug
-

2.4 Declaring a Variable

JavaScript provides three keywords for declaring variables:

var

let

const

Example:

```
let age;
```

Here we have **declared** a variable.

We have not assigned a value yet.

2.5 Initialization

Giving a value to a variable is called **initialization**.

```
let age = 22;
```

Here:

Declaration + Initialization

happen together.

Another example:

```
let name;
```

```
name = "Rahul";
```

The first line declares the variable.

The second line assigns a value.

2.6 Assignment

The assignment operator is:

=

Example:

```
let age = 22;
```

It means:

Store the value 22 in the variable age.

It does **not** mean mathematical equality.

2.7 var

var is the older way of declaring variables in JavaScript.

Example:

```
var name = "Rahul";
```

```
console.log(name);
```

Output:

Rahul

A var variable can be reassigned:

```
var age = 20;
```

```
age = 21;
```

```
console.log(age);
```

Output:

21

It can also be redeclared in the same scope:

```
var x = 10;
```

```
var x = 20;
```

```
console.log(x);
```

Output:

20

2.8 let

let was introduced with **ES6**.

Example:

```
let age = 20;
```

```
age = 21;
```

```
console.log(age);
```

Output:

21

A let variable can be reassigned.

However:

```
let x = 10;
```

```
let x = 20;
```

in the same scope causes an error.

Therefore:

let → can be reassigned

let → cannot be redeclared in the same scope

2.9 const

const is used when the variable binding should not be reassigned.

Example:

```
const pi = 3.14159;
```

```
console.log(pi);
```

This is invalid:

```
const pi = 3.14159;
```

```
pi = 3.14;
```

It produces an error because the binding cannot be reassigned.

Important

const does **not** mean that every object or array stored in the variable becomes immutable.

For example:

```
const student = {  
  name: "Rahul"  
};
```

```
student.name = "Amit";
```

This is allowed because the object itself can be modified; the variable student cannot be assigned to a different object.

2.10 var vs let vs const

This is an important examination question.

Feature	var	let	const
Introduced	Older JS	ES6	ES6
Reassign	Yes	Yes	No
Redeclare same scope	Yes	No	No
Block scoped	No	Yes	Yes
Must initialize immediately	No	No	Yes
Recommended for modern code	Usually no	Yes	Yes

Simple rule for students

Use:

const

by default, when reassignment isn't required.

Use:

let

when the value needs to change.

Avoid using:

var

in modern JavaScript unless there is a specific reason to use its older scoping behavior.

2.11 Scope

Scope determines where a variable can be accessed.

There are different types of scope, including:

- Global scope
- Function scope
- Block scope

For this chapter, students should understand the basic difference between var and let/const.

Block Scope

A block is generally code enclosed by { }.

Example:

```
{  
  let x = 10;  
  console.log(x);  
}
```

x exists inside that block.

But:

```
{  
  let x = 10;  
}
```

```
console.log(x);
```

will produce an error because x is not accessible outside the block.

The same applies to const.

2.12 Variable Naming Rules

JavaScript variable names follow certain rules.

Rule 1

A variable name can contain:

- Letters
- Digits
- _
- \$

Example:

```
let studentName;
```

```
let student_age;
```

```
let $price;
```

```
let marks1;
```

Rule 2

A variable name cannot start with a digit.

Invalid:

```
let 1student;
```

Valid:

```
let student1;
```

Rule 3

Spaces are not allowed.

Invalid:

```
let student name;
```

Valid:

```
let studentName;
```

Rule 4

JavaScript is case-sensitive.

These are different:

```
let name;
```

```
let Name;
```

```
let NAME;
```

Rule 5

Reserved keywords cannot normally be used as variable names.

For example:

```
let class;
```

is invalid because class is a JavaScript keyword.

Other keywords include:

if

else

for

while

function

return

class

const

let
var
new
this

2.13 Naming Convention

For JavaScript variables, **camelCase** is commonly used.

Example:

```
let studentName;  
let employeeSalary;  
let totalMarks;  
let mobileNumber;
```

Avoid:

```
let student_name;  
unless your project's coding convention specifically uses snake_case.
```

2.14 JavaScript Data Types

A **data type** tells us what kind of value a variable contains.

Examples:

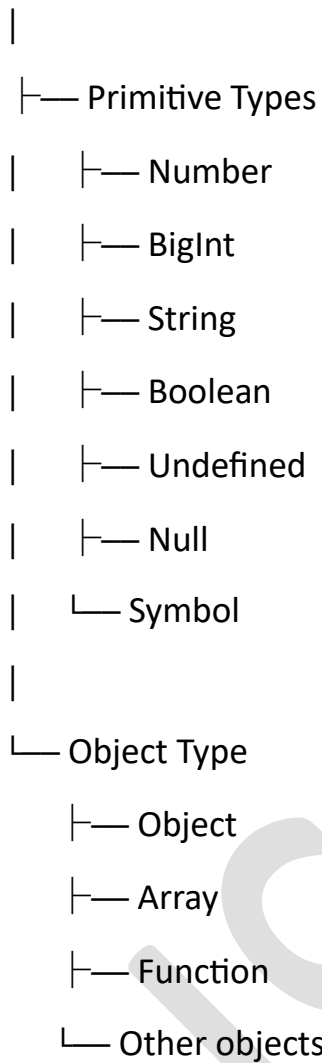
```
let age = 22;  
let name = "Rahul";  
let passed = true;
```

The values have different types.

JavaScript has **primitive** and **object/reference** types.

2.15 JavaScript Data Type Classification

JavaScript Data Types



2.16 Number

JavaScript uses the number type for most numeric values.

Example:

```
let age = 22;
```

```
let marks = 87.5;
```

```
let temperature = -5;
```

All are number.

```
console.log(typeof age);
```

Output:

number

JavaScript's number uses IEEE 754 double-precision floating-point representation.

2.17 Integer and Decimal

Unlike languages such as Java, JavaScript does not have separate primitive types called int and float.

Both are generally:

number

Example:

```
let a = 10;
```

```
let b = 10.5;
```

```
console.log(typeof a);
```

```
console.log(typeof b);
```

Output:

number

number

2.18 Special Number Values

JavaScript also has special numeric values.

Infinity

```
console.log(10 / 0);
```

Output:

Infinity

NaN

NaN means:

Not a Number

Example:

```
console.log("Hello" * 5);
```

Output:

NaN

Interestingly:

typeof NaN

returns:

number

This is a common viva question.

2.19 BigInt

BigInt is used for integers larger than the safe integer range of JavaScript's Number type.

Example:

```
let bigNumber = 123456789012345678901234567890n;
```

```
console.log(bigNumber);
```

Notice the:

n

at the end.

```
console.log(typeof bigNumber);
```

Output:

bigint

2.20 String

A string represents text.

Example:

```
let name = "Rahul";
```

Strings can use:

Double quotes

```
let city = "Bhubaneswar";
```

Single quotes

```
let city = 'Bhubaneswar';
```

Template literals

```
let city = `Bhubaneswar`;
```

Template literals become particularly useful in later chapters.

2.21 Boolean

Boolean has only two possible values:

true

false

Example:

```
let isLoggedIn = true;
```

```
let isAdmin = false;
```

Boolean values are heavily used in:

- Conditions
 - Loops
 - Validation
 - Authentication
 - Decision-making
-

2.22 Undefined

A variable that has been declared but has not been assigned a value has the value:

undefined

Example:

```
let age;
```

```
console.log(age);
```

Output:

undefined

And:

```
console.log(typeof age);
```

Output:

undefined

2.23 Null

null represents an intentional absence of an object/value.

Example:

```
let selectedStudent = null;
```

This means:

Currently, no student is selected.

Important difference

```
let a;
```

means:

undefined

while:

```
let b = null;
```

means:

intentional empty/no value

2.24 typeof Operator

typeof is used to determine the type of a value.

Example:

```
let name = "Rahul";
```

```
let age = 22;
```

```
let passed = true;
```

```
console.log(typeof name);
```

```
console.log(typeof age);
```

```
console.log(typeof passed);
```

Output:

string

number

boolean

More examples:

```
console.log(typeof undefined);
```

```
console.log(typeof 100);
```

```
console.log(typeof "Hello");
```

```
console.log(typeof true);
```

```
console.log(typeof 10n);
```

Output:

undefined

number

string

boolean

bigint

2.25 Important typeof Special Case

Students should remember:

typeof null

returns:

object

This is a historical behavior of JavaScript and is not the same as saying that null is an object.

2.26 Objects

Objects store collections of related data.

Example:

```
let student = {  
  name: "Rahul",  
  age: 22,  
  course: "MCA"  
};
```

We can access properties:

```
console.log(student.name);
```

```
console.log(student.age);
```

Output:

Rahul

22

Objects will be studied in much greater detail in a later chapter.

2.27 Arrays

Arrays store multiple values.

Example:

```
let marks = [80, 75, 90, 85];
```

Access an element:

```
console.log(marks[0]);
```

Output:

80

Remember:

Array indexing starts from 0.

Arrays will have their own chapter later.

2.28 Functions

Functions are reusable blocks of code.

Example:

```
function greet() {  
    console.log("Hello");  
}
```

A function is also an object in JavaScript.

Students will study functions separately.

2.29 Primitive vs Object Types

Primitive values

The seven primitive data types are:

1. String
2. Number

- 3. BigInt
- 4. Boolean
- 5. Undefined
- 6. Null
- 7. Symbol

Object type

Examples include:

Object

Array

Function

Date

Map

Set

RegExp

2.30 Dynamic Typing

JavaScript is dynamically typed.

This means a variable does not have to permanently hold values of one particular type.

Example:

```
let value = 100;
```

```
console.log(typeof value);
```

```
value = "Hello";
```

```
console.log(typeof value);
```

```
value = true;
```

```
console.log(typeof value);
```

Output:

number

string

boolean

The variable's current value determines its runtime type.

2.31 Type Conversion

Sometimes we need to convert one data type into another.

For example:

String → Number

Number → String

String → Boolean

There are two broad concepts:

Type Conversion

|
├— Explicit conversion

|
└— Implicit coercion

2.32 String to Number

Number()

```
let value = "100";
```

```
let number = Number(value);
```

```
console.log(number);
```

```
console.log(typeof number);
```

Output:

100

number

parseInt()

```
let value = "100";
```

```
let number = parseInt(value);
```

```
console.log(number);
```

Output:

100

It is commonly used to parse an integer from a string.

parseFloat()

```
let value = "10.50";
```

```
let number = parseFloat(value);
```

```
console.log(number);
```

Output:

2.33 Number to String

Use:

String()

Example:

```
let number = 100;
```

```
let value = String(number);
```

```
console.log(value);
```

```
console.log(typeof value);
```

Output:

100

string

Another approach:

```
let number = 100;
```

```
let value = number.toString();
```

2.34 Boolean Conversion

Use:

Boolean()

Example:

```
console.log(Boolean(1));
```

```
console.log(Boolean(0));
```

Output:

true

false

Some common falsy values include:

false

0

-0

0n

""

null

undefined

NaN

Most other values are truthy.

2.35 Implicit Type Conversion

JavaScript can automatically convert values in certain operations.

Example:

```
let result = "10" + 5;
```

```
console.log(result);
```

Output:

105

Why?

Because + with a string can perform string concatenation.

But:

```
let result = "10" - 5;
```

```
console.log(result);
```

Output:

5

JavaScript converts "10" to a number for the subtraction operation.

This behavior is called **type coercion**.

2.36 A Very Important Example

```
console.log("5" + 2);
```

```
console.log("5" - 2);
```

```
console.log("5" * 2);
```

```
console.log("5" / 2);
```

Output:

52

3

10

2.5

Explain this carefully to students because it is frequently asked in JavaScript interviews and viva examinations.

2.37 == vs ===

This topic connects directly with type conversion.

==

Loose equality may perform type coercion.

```
console.log(5 == "5");
```

Output:

true

===

Strict equality checks both value and type.

```
console.log(5 === "5");
```

Output:

false

Recommended practice

Teach students to generally prefer:

===

and:

!==

when they want strict comparisons.

PART B — PRACTICAL CLASS

Practical 2.1 — Create Variables

```
let name = "Rahul";
```

```
let age = 22;
```

```
let marks = 85.5;
```

```
console.log(name);
```

```
console.log(age);
```

```
console.log(marks);
```

Practical 2.2 — Student Information

```
let studentName = "Amit";
```

```
let rollNumber = 101;
```

```
let course = "MCA";
```

```
let semester = 3;
```

```
console.log("Student Name:", studentName);
```

```
console.log("Roll Number:", rollNumber);
```

```
console.log("Course:", course);
```

```
console.log("Semester:", semester);
```

Practical 2.3 — Calculate Total Marks

```
let java = 85;
```

```
let dbms = 80;
```

```
let web = 90;
```

```
let os = 75;
```

```
let total = java + dbms + web + os;
```

```
console.log("Total Marks:", total);
```

Practical 2.4 — Calculate Average

```
let java = 85;
```

```
let dbms = 80;
```

```
let web = 90;
```

```
let os = 75;
```

```
let total = java + dbms + web + os;
```

```
let average = total / 4;
```

```
console.log("Total:", total);  
console.log("Average:", average);
```

Practical 2.5 — Demonstrate Data Types

```
let name = "Rahul";  
let age = 22;  
let percentage = 85.5;  
let passed = true;  
let address;  
let data = null;  
let bigNumber = 12345678901234567890n;
```

```
console.log(typeof name);  
console.log(typeof age);  
console.log(typeof percentage);  
console.log(typeof passed);  
console.log(typeof address);  
console.log(typeof data);  
console.log(typeof bigNumber);
```

Ask students to predict the output **before executing it**.

Practical 2.6 — Type Conversion

```
let value = "100";  
  
let number = Number(value);
```

```
console.log(number);  
console.log(typeof number);
```

Practical 2.7 — Number to String

```
let number = 500;  
  
let value = String(number);
```

```
console.log(value);  
console.log(typeof value);
```

Practical 2.8 — Boolean Conversion

```
console.log(Boolean(1));  
console.log(Boolean(0));  
console.log(Boolean("Hello"));  
console.log(Boolean(""));  
console.log(Boolean(null));  
console.log(Boolean(undefined));  
Ask students to predict every output.
```

Practical 2.9 — == and ===

```
console.log(10 == "10");  
  
console.log(10 === "10");  
Expected:
```

true

false

Explain **why**, not just the output.

Practical 2.10 — Dynamic Typing

```
let value = 100;
```

```
console.log(value);
```

```
console.log(typeof value);
```

```
value = "Hello";
```

```
console.log(value);
```

```
console.log(typeof value);
```

```
value = true;
```

```
console.log(value);
```

```
console.log(typeof value);
```

Practical 2.11 — Simple Bill Calculation

```
let price = 500;
```

```
let quantity = 3;
```

```
let total = price * quantity;
```

```
console.log("Price:", price);  
console.log("Quantity:", quantity);  
console.log("Total:", total);
```

Practical 2.12 — Employee Salary

```
let basicSalary = 25000;  
let hra = 5000;  
let allowance = 3000;  
  
let totalSalary = basicSalary + hra + allowance;  
  
console.log("Basic Salary:", basicSalary);  
console.log("HRA:", hra);  
console.log("Allowance:", allowance);  
console.log("Total Salary:", totalSalary);
```

Practical 2.13 — Celsius to Fahrenheit

Formula:

$$F = (C \times 9/5) + 32$$

JavaScript:

```
let celsius = 30;  
  
let fahrenheit = (celsius * 9 / 5) + 32;  
  
console.log("Celsius:", celsius);  
console.log("Fahrenheit:", fahrenheit);
```

Practical 2.14 — Rectangle Area

Formula:

Area = Length × Breadth

let length = 10;

let breadth = 5;

let area = length * breadth;

console.log("Area:", area);

Practical 2.15 — Circle Area

Formula:

Area = πr^2

let radius = 5;

let area = Math.PI * radius * radius;

console.log("Area:", area);

Math will be covered in more detail later.

PRACTICAL ASSIGNMENTS

Give these to students after the practical class.

Assignment 1

Create variables for:

Student Name

Roll Number

Course

Semester

College

Age

and display them.

Assignment 2

Create a program to calculate:

Total marks

Average marks

for five subjects.

Assignment 3

Create a program to calculate:

Simple Interest

Formula:

$$SI = (P \times R \times T) / 100$$

Assignment 4

Create a program to calculate the area of:

- Circle
- Rectangle
- Triangle
- Square

Assignment 5

Create a program demonstrating all seven primitive data types.

Assignment 6

Write a program demonstrating the difference between:

==

and:

===

Assignment 7

Write a program demonstrating dynamic typing.

Viva Questions

1. What is a variable?
2. What is the difference between declaration and initialization?
3. What are var, let, and const?
4. Which keyword should generally be used when reassignment isn't required?
5. Can a const variable be reassigned?
6. Can let be redeclared in the same scope?
7. Is JavaScript statically or dynamically typed?
8. How many primitive data types are there?
9. What is undefined?
10. What is null?
11. What is the difference between null and undefined?
12. What is typeof?
13. What is the type of NaN?
14. What is the type of null according to typeof?
15. What is BigInt?
16. What is type conversion?
17. What is type coercion?
18. Difference between == and ===?
19. What is block scope?

20. Why is var generally avoided in modern JavaScript?

Important MCQs

1. Which keyword creates a block-scoped variable?

- A. var
- B. let
- C. define
- D. variable

Answer: B

2. Which keyword prevents reassignment of the variable binding?

- A. var
- B. let
- C. const
- D. static

Answer: C

3. What is the type of "Hello"?

- A. number
- B. boolean
- C. string
- D. object

Answer: C

4. What is the output?

`console.log(typeof 10);`

- A. integer
- B. number
- C. float
- D. numeric

Answer: B

5. What is the output?

```
console.log(5 == "5");
```

- A. true
- B. false
- C. error
- D. undefined

Answer: A

6. What is the output?

```
console.log(5 === "5");
```

- A. true
- B. false
- C. error
- D. null

Answer: B

7. Which represents an intentionally empty value?

- A. undefined
- B. null
- C. empty
- D. void

Answer: B

8. Which suffix is used for a BigInt literal?

- A. b
- B. i
- C. n
- D. big

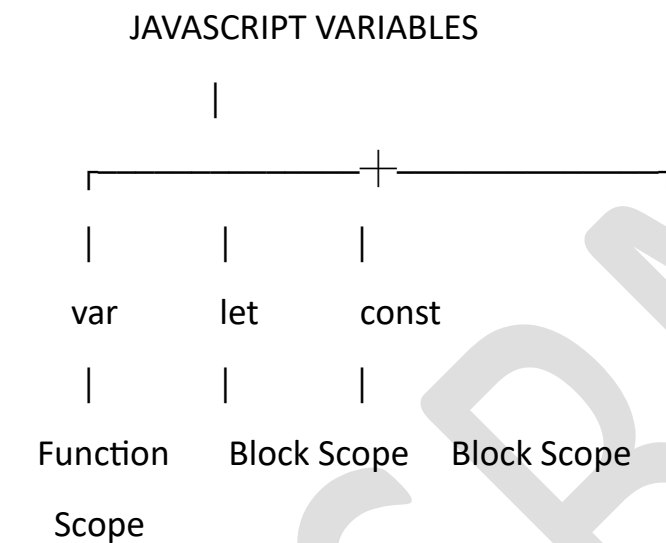
Answer: C

Example:

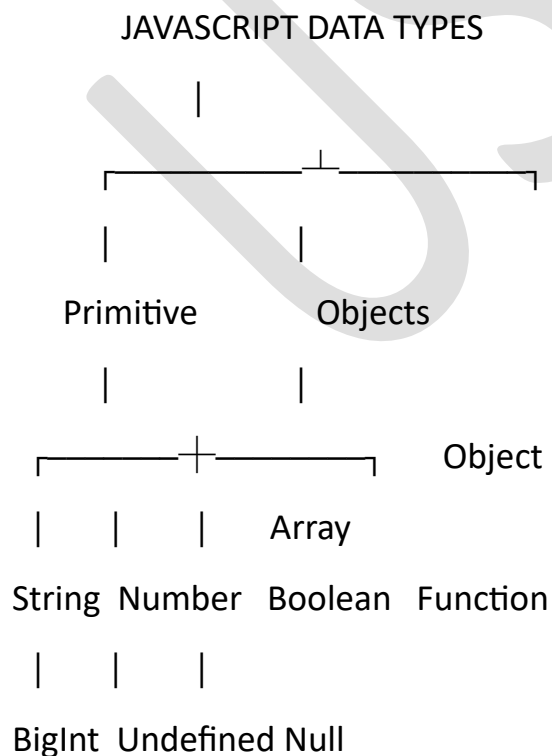
123456789n

Board/Notebook Summary

At the end of the class, have students write this:



And:



Symbol

One-line revision

Variable = named storage/reference for a value

let = reassignable, block-scoped binding

const = non-reassignable binding, block-scoped

var = older, function-scoped declaration

Data type = category/type of a value

JavaScript = dynamically typed

typeof = operator used to inspect the runtime type

== = loose equality

=== = strict equality